

Projet Pédagogique : Clean Code & Clean Architecture

(2026)

Introduction

EcoEats, une start-up française ambitieuse, souhaite concurrencer les géants de la livraison de repas (UberEatss, Deliverooo) en proposant une alternative éthique et transparente. La plateforme a pour objectif de connecter les **Clients**, les **Restaurateurs** et les **Livreurs** tout en garantissant une répartition équitable des revenus et une optimisation des trajets pour réduire l'empreinte carbone.

Vous avez été recruté pour concevoir le cœur de cette plateforme en assurant une flexibilité totale face aux changements technologiques futurs.

Fonctionnalités (15 points)

1. Client (La commande)

- **Navigation et Panier :**
 - En tant que client, je dois pouvoir parcourir les menus des restaurants disponibles.
 - Je dois pouvoir constituer un panier. **Règle métier :** Un panier ne peut contenir des articles que d'un seul restaurant à la fois. Si j'ajoute un article d'un autre restaurant, le système doit me proposer de vider le panier actuel ou d'annuler l'action.
- **Passage de commande :**
 - Je dois pouvoir valider ma commande. Le prix total doit inclure le prix des plats, les frais de livraison (calculés en fonction de la distance à vol d'oiseau) et les frais de service de la plateforme.
 - Une fois la commande payée (simulation), une facture détaillée est générée.

2. Restaurateur (La préparation)

- **Gestion du Menu :**
 - En tant que restaurateur, je dois pouvoir ajouter, modifier ou supprimer des plats (nom, description, prix, allergènes).

- Je dois pouvoir définir un "Stock journalier" pour chaque plat. Si le stock est à 0, le plat n'est plus commandable.
- **Workflow de commande :**
 - Je reçois les commandes en temps réel. Je dois pouvoir **Accepter** ou **Refuser** une commande.
 - Si j'accepte, je dois indiquer un temps de préparation estimé. Une fois prêt, je change le statut de la commande à "Prête pour collecte".

3. Livreur (La logistique)

- **Attribution et Livraison :**
 - En tant que livreur, je peux me déclarer "Disponible" ou "Indisponible".
 - Je reçois des propositions de livraison (uniquement pour les commandes "Prêtes" ou en cours de préparation).
 - **Règle métier :** Un livreur ne peut accepter qu'une seule livraison à la fois (sauf s'il possède un statut "Expert", auquel cas il peut en cumuler deux du même restaurant).
- **Revenus :**
 - À chaque livraison terminée, mon portefeuille virtuel est crédité.
 - Le montant est calculé selon une formule fixe (Prise en charge + Prix au km + Pourboire intégral du client). La plateforme ne prend aucune commission sur la part du livreur.

Contraintes Techniques

1. Langage

- Développement en **TypeScript** (Backend et Frontend).

2. Clean Architecture

- **Séparation stricte des couches :**
 - **Domain** (Entities, Value Objects) : Cœur métier pur, sans aucune dépendance externe.
 - **Application** (Use Cases, Ports) : Orchestration des règles métier.
 - **Interface** (Controllers, Presenters) : Point d'entrée de l'API ou de l'UI.
 - **Infrastructure** (Repositories implémentés, Frameworks, DB drivers).
- Chaque couche ne doit dépendre que des couches plus internes (Règle de dépendance).

3. Indépendance Technologique (Le "Plug & Play")

Afin de prouver la robustesse de votre architecture, vous devez implémenter :

- **2 adaptateurs** (in-memory, SQL, NoSQL, etc) pour les bases de données et **2 frameworks** backend (Nest.js, Express, Fastify, etc).

4. Clean Code

- Respect des principes **SOLID**.
 - Nommage explicite des variables et fonctions.
 - Fonctions courtes et à responsabilité unique.
 - Gestion des erreurs via des Exceptions typées ou des retours de type Result (Monad).
 - Les pratiques issues des ouvrages de **Robert C. Martin (Uncle Bob)** sont attendues.
-

Bonus

1. CQRS (Command Query Responsibility Segregation)

- a. Séparer strictement les modèles d'écriture (Commands) et les modèles de lecture (Queries).
- b. Permet de préparer l'Event-Sourcing.

2. Event-Sourcing

- a. Au lieu de stocker l'état actuel d'une commande (ex: "LIVRÉE"), stocker la suite d'événements (*OrderCreated*, *OrderPaid*, *OrderPrepared*, *OrderPickedUp*, *OrderDelivered*).
- b. Permettre de "rejouer" l'historique pour savoir exactement où se trouvait le livreur à un instant T.
- c. Utilisation de microservices bienvenue.

3. Framework Frontend

- a. Utilisation de plusieurs frameworks frontend pour consommer la même API.
- b. **Angular, React & Solid.js** à privilégier.
- c. Lister les avantages et inconvénients de chacun (notamment sur la gestion d'état et la réactivité).